DATA ARCHITECTURE & INGESTION BLUEPRINT

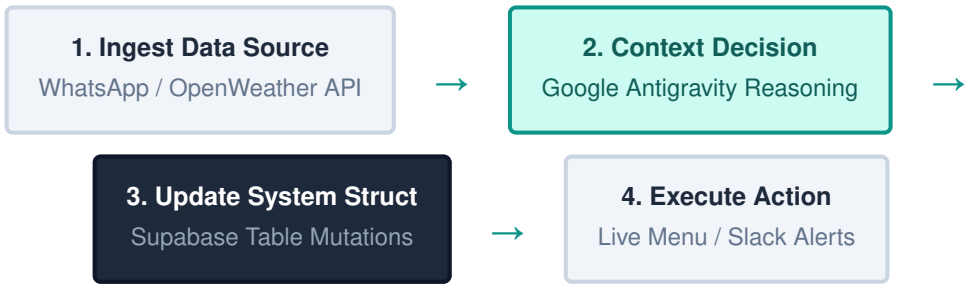
MenuMind: Complete End-to-End Operational Lifecycle

Mapping Unstructured External Signals to Autonomous Database States and Live Mobile Executions

1. Introduction & Core Concept Model

The MenuMind engine fulfills the strict mandates of **Challenge 1: Autonomous Content-to-Action Agent** by translating real-time business chaos into immediate, rule-governed menu adjustments. This architectural guide breaks down exactly where data is harvested, how logic transitions across system layers, how the underlying data structures (the `struct` schemas) are mutated, and how final executions are visually dispatched.

Your structural planning notes outline a classical computer science linear progression pattern. The system matches that lifecycle precisely:



2. Data Ingestion Architecture: Where Data is Fetched From

To ensure complete operational insight without expensive corporate integrations, MenuMind isolates three primary external operational data streams:

Data Source Stream	Fetch Mechanism	Raw Formats Ingested	Operational Relevance
Supplier Communications	Manual paste form / Inbound HTTP Webhooks	Unstructured text strings, mixed English and Roman Urdu (e.g., *"Aziiz bhai, chicken short hai market me today."*)	Identifies physical product stock outages, quality rejections, or immediate wholesale delivery distribution failures.
Environmental Context	Automated API Call (OpenWeatherMap)	Structured JSON streams extracted automatically via Make.com timed scenario nodes.	Tracks critical heat waves, monsoonal flash flood warnings, or storms impacting customer seating footprints.
Competitive Intelligence	Manual text logging / Captioned images	Pasted text references or competitor digital flyers detailing sudden price inflation.	Signals major local market price movements, preventing margin loss when supply chain costs spike across town.

3. The Orchestration Brain: Google Antigravity Logic

Once Make.com aggregates the active operational inputs, it compiles them into a unified execution payload. This consolidated text block is transmitted via an HTTP POST request to the **Google Antigravity Brain** for deep semantic reasoning. The agent performs a single-pass execution to evaluate data meaning through a series of logical modules:

1. **Entity Parser Module:** Strips colloquialisms and extracts core operational tokens (e.g., identifies that "chicken wrap" is the active target item).
2. **Contradiction Engine:** cross-references incoming notifications against current states to resolve conflicts. For example, if a supplier states an item is unavailable, but local inventory tools show a physical buffer stock, it computes whether to execute immediate caution or allow short-term sales.
3. **Ethical Safety Shield:** If keywords corresponding to crises (e.g., *flood, strike, municipal disaster*) are discovered, the brain enforces a zero-surge pricing constraint, ensuring brand reputation protection.
4. **Cascade Planner:** Generates a primary task alongside up to two coordinated supporting tasks to maintain operational harmony.

4. The System Struct: Database Models & Schemas

Following the precise model outlined in your planning notes, the data passes from the *Decision Engine* directly into database models. These represent the structured schema (`struct`) inside **Supabase** that governs the live application frontend:

Table 1: Menu Items Schema (`struct menu_items`)

```
CREATE TABLE menu_items (
    id VARCHAR PRIMARY KEY,            -- E.g., 'chicken_wrap'
    name VARCHAR NOT NULL,             -- Item display name
    base_price NUMERIC(10,2),          -- Baseline price
    current_price NUMERIC(10,2),       -- Adaptive operational price
    is_available BOOLEAN DEFAULT true, -- Active availability switch
    shipping_buffer_days INT,          -- Operational restock lag time
    updated_at TIMESTAMP DEFAULT NOW()
);
```

Table 2: Agent Orchestration History (`struct agent_logs`)

```
CREATE TABLE agent_logs (
    id BIGSERIAL PRIMARY KEY,
    timestamp TIMESTAMP DEFAULT NOW(),
    raw_signals TEXT,                  -- Incoming unstructured string dump
    reasoning_trace JSONB,             -- Step-by-step trace array output
    confidence_score NUMERIC(3,2),     -- Confidence coefficient (0.00 to 1.00)
    requires_approval BOOLEAN          -- Safety validation queue flag
);
```

5. Execution Phase: Downstream Action Simulation

The final leg of the system transforms the mutated database models into physical business updates. Make.com watches for changes in the Supabase state variables and handles the following automated actions:

- **Live Dashboard Updates:** The FlutterFlow mobile interface re-renders automatically as it listens to active Supabase stream triggers. Unavailable items disappear cleanly, or modified prices show an immediate visual adjustment badge.
- **Kitchen Notification Dispatches:** An inbound webhook fires an instantaneous alert directly into the back-of-house Slack team channel, alerting line cooks regarding immediate recipe modifications or item suspensions.
- **Customer Notice Automations:** Transactional emails dispatch via EmailJS templates to alert delivery customers of sudden menu adjustments, proactively offering automated alternatives to protect sales volume.

> **Technical Polish Summary:** By partitioning the system into clear steps—Ingest (Make.com API layers), Reason (Google Antigravity Single-Pass), Struct (Supabase relational data arrays), and Execute (FlutterFlow state views and notifications)—the build remains incredibly stable, robust against edge cases, and completely executable inside a standard 3-day hackathon sprint.